

Infinite Gaussian Mixture Autoencoder for Adaptive Deep Unsupervised Clustering

Ying Li , Junyu Xuan , *Senior Member, IEEE*, and Richard Yida Xu , *Member, IEEE*

Abstract—Clustering is a fundamental task in machine learning and data mining. The success of deep learning, especially deep generative models, has given birth to the next generation of clustering—deep unsupervised clustering featured by the capability of clustering large-scale and high-dimensional data. One typical example is the Gaussian mixture variational autoencoder (GMVAE), where the prior of the latent variable is a mixture of Gaussian distributions (GMMs). However, the mixture number manually given in GMVAE greatly impacts on the final clustering performance and also limits the adaptation ability when new clusters come. To solve this issue, we propose an infinite Gaussian mixture autoencoder (IGMVAE), which replaces the plain Gaussian prior in VAE with an infinite Gaussian mixture distribution. Our method adaptively learns the number of mixtures from data, overcoming the manual setting limitation in GMVAE, and dynamically expands clusters for novel categories. Experimental results on three datasets show that our proposed IGMVAE achieves 5.5%–14.25% higher clustering accuracy compared to GMVAE. Moreover, our model is capable of generating high-quality data in a fully unsupervised manner.

Impact Statement—This work presents a significant advancement in the field of unsupervised learning by introducing the IGMVAE, a novel variant of the variational autoencoder tailored for adaptive deep clustering. Unlike conventional methods that require predefined cluster numbers or struggle with dynamic data, IGMVAE leverages a nonparametric Bayesian prior to model the latent space as an infinite Gaussian mixture, enabling automatic adaptation to evolving data distributions and emerging categories. This key innovation bridges the gap between flexible deep generative models and scalable, data-driven clustering. Our model not only achieves superior clustering performance on high-dimensional data but also enables categorywise sample generation

without relying on any labeled supervision. This ability to generate class-conditional samples in a fully unsupervised manner is particularly valuable in domains where labeled data is scarce or unavailable.

Index Terms—Data generation, deep clustering, infinite Gaussian mixture model (IGMM), unsupervised learning, variational autoencoder (VAE).

I. INTRODUCTION

CLUSTERING remains a fundamental and important task in machine learning research and often serves as a critical preprocessing step in various data mining tasks, including anomaly detection [1], [2], domain adaptation [3], [4], and discriminative representation learning [5], [6], etc. Clustering groups data instances into clusters. These clusters of samples provide a global characterization of data instances, offering significant benefits for various subsequent analyses on the entire dataset.

Traditional clustering methods, such as centroid-based K-means [7] and density-based spatial clustering [8], primarily rely on similarity measures between data instances in the original data space, which normally exhibits high-dimensional and intricate features that make the clustering difficult. Apart from these two categories, there is also a branch of methods clustering data instances via fitting distribution, such as Gaussian mixture models (GMMs) [9]. This distribution-based method provides a probabilistic estimate of the likelihood that each data point belongs to each cluster rather than a hard assignment, and quantifies the inherent uncertainty, which is useful for some following safety-critical tasks. The accompanying disadvantage is the expensive computation, especially for the high-dimensional and intricate data, and the hard distributional form assumption selection.

In recent years, deep learning has achieved success in representation learning, especially in unstructured and high-dimensional data. With its development, deep clustering, which is the joint optimization of deep representation learning and clustering, has attracted more and more attention. It enhances clustering by leveraging the powerful representation capabilities of deep learning. Because of deep learning's ability to extract features from complex, high-dimensional data, we map the data into latent space and then perform clustering in that space. This reduces the difficulty of clustering directly on the raw data, allowing us to discover underlying structures more effectively [10], [11]. In particular, distribution-based clustering

Received 2 August 2025; revised 29 September 2025; accepted 16 March 2026. Date of publication 24 March 2026; date of current version 31 August 2026. This work was supported in part by the R&D Program of Beijing Municipal Education Commission under Grant 10051360024XN151-41 and in part by Beijing Natural Science Foundation under Grant L253018. This article was recommended for publication by Associate Editor Pierpaolo D'Urso upon evaluation of the reviewers' comments. (*Corresponding author: Ying Li.*)

Ying Li is with the School of Artificial Intelligence and Computer Science, North China University of Technology, Beijing 100144, China (e-mail: ying.li@ncut.edu.cn).

Junyu Xuan is with the Faculty of Engineering and Information Technology, University of Technology Sydney, Ultimo, NSW 2007, Australia (e-mail: junyu.xuan@uts.edu.au).

Richard Yida Xu is with the Department of Mathematics, Hong Kong Baptist University, Hong Kong (e-mail: xuyida@hkbu.edu.hk).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2026.3677237>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3677237

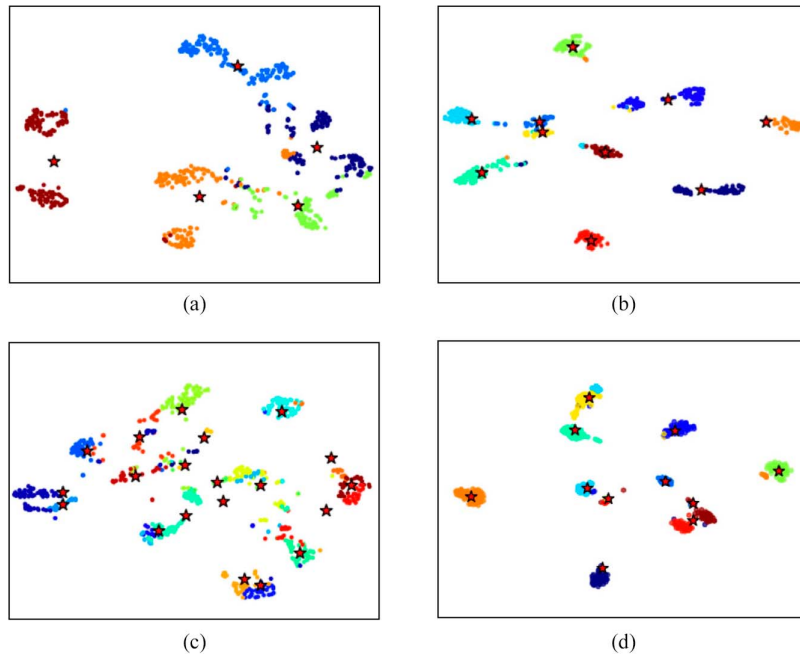


Fig. 1. T-SNE graph of the latent variables on YouTube traffic dataset. (a) GMVAE learns the latent variables with mixture number 5. (b) GMVAE learns the latent variables with mixture number 10. (c) GMVAE learns the latent variables with mixture number 20. (d) Latent variables of IGMVAE (our model).

is boosted by the unsupervised deep learning producing deep generative models that offer a strong capability to estimate the density of observed data without any labels, especially high-dimensional and complicated ones. One typical example is the Gaussian mixture variational autoencoder (GMVAE) [12], [13], which is built on a deep generative model—variational autoencoder (VAE) [14], [15]. GMVAE replaces the prior for the latent variables in VAE with a mixture of Gaussian distributions (GMMs) to achieve deep clustering, which has been recognized as an important benchmark of deep unsupervised clustering.

However, this method still has some limitations. First, the mixture number must be set manually and the clustering performance varies significantly under different mixture numbers. As shown in Fig. 1, if the mixture number is set to be the same as the true number of clusters, the performance is good. The greater the difference, the worse the clustering performance will be. Second, as GMVAE fixes the number of clusters in advance, if there are new categories of data coming, it cannot identify new clusters, and it cannot make adaptive adjustments based on the novel data.

To resolve these limitations, we propose an adaptive deep unsupervised clustering method, namely, infinite Gaussian mixture autoencoder (IGMVAE). IGMVAE replaces the fixed Gaussian prior with an infinite Gaussian mixture model (IGMM), enabling the model to infer the number of clusters directly from the data.

The theoretical foundation of IGMM lies in its nonparametric Bayesian nature, which allows for a potentially unbounded number of mixture components. This flexibility is achieved through the use of a Dirichlet process (DP) prior, which governs

the distribution over cluster assignments. Unlike parametric models, the DP does not require a fixed number of clusters; instead, it defines a probability distribution over countably infinite partitions, allowing the model to adaptively allocate clusters as needed. Introducing such a flexible prior is not easy because it brings additional challenges to the model inference, e.g., how to adaptively change the cluster number and the corresponding neural networks. Therefore, we propose a particularly designed model inference algorithm based on the stick-breaking representation of the DP and a genius neural network architecture to facilitate adaptive clustering. The contributions of our study are summarized as follows.

- 1) We propose a new adaptive deep unsupervised clustering method, which is able to adapt to incremental data and new clusters. To the best of our knowledge, this is the first work that can do adaptive clustering in an unsupervised manner.
- 2) A new variant of VAE with latent variables satisfying an infinite Gaussian mixture distribution is developed. Compared with other VAE variants, ours can greatly improve the data adaptation ability.
- 3) Our model can not only achieve better clustering performance for high-dimensional data but also facilitate the data generation by category without any supervised information.

In the following sections, we first introduce the related work on deep unsupervised clustering and VAE in Section II, and then the preliminary knowledge is given in Section III. Our method is described in Section IV, and finally, our results and ablation study are analyzed in Section V. Section VI concludes this article.

II. RELATED WORK

In this section, we first introduce the recent deep learning-based cluster methods that are used to learn latent features of the training data. Second, some generative based cluster methods have been proposed and as our proposed model is based on a generative model, VAE, we then introduce some basic ideas of VAE and its variants.

A. Deep Unsupervised Clustering

The main purpose of a deep clustering method is to learn the latent space of observed data through deep learning methods and discover the structure of the latent space. Deep embedded clustering (DEC) [16] is an approach that combines the learning of feature representations and cluster assignments through deep neural networks. It accomplishes this by training a mapping from the data space to a lower dimensional feature space, where it iteratively refines a clustering objective. Information maximizing self-augmented training (IMSAT) [17] employs data augmentation to enforce invariance in discrete representations. To do this, it promotes the similarity between the predicted representations of augmented data points and the original data points and enhances the information-theoretic dependence between the data points and their predicted discrete representations. Spectral Net [18] learns a map that embeds input data points into the eigenspace of their associated graph Laplacian matrix and subsequently clusters them.

In addition, some deep clustering methods based on generative models have also been proposed. Most are based on VAE, which replaces the prior of the latent variable with some specific distribution, such as Gaussian mixture distribution. Specifically, Kingma et al. [13] proposed a stacked generative semisupervised model based on the fact that their prior distribution is a neural network transformation of both continuous and discrete variables, with Gaussian and categorical priors, respectively. GMVAE [12] replaces the prior of the latent space of the observed data with a mixture of Gaussians, based on the assumption that different clusters of data are generated from different Gaussian distributions. Deep clustering by Gaussian mixture variational autoencoder with graph embedding [19] similarly regards GMM as a prior distribution for potential features, combining local structural constraints and global insights based on sample similarity. Latent tree variational autoencoder [20] leverages latent tree structures to discover hierarchical clusters. Prasad et al. [21] introduces an innovative unsupervised image clustering algorithm, variational autoencoder for image clustering (VAEIC). Diverging from GMVAE, VAEIC opts for a deterministic prior distribution, discarding random variables. Dual variational autoencoder [22], which employs a von Mises–Fisher mixture prior with a dual-VAE architecture. Beyond GMM priors, MVAE [23] combines multiple VAEs with the d-variable Hilbert–Schmidt independence criterion to partition latent space and adaptively determine model capacity. Ji et al. [24] proposed decoder-free variational clustering (DFVC), a VAE-based clustering method that relies on mutual information. Unlike prior encoder–decoder frameworks, DFVC removes the decoder to improve model depth and learning capacity,

motivated by the link between reconstruction error minimization and mutual information maximization. To further enhance interpretability. Liu et al. [25] introduced Cloud-VAE, which employs a cloud-model-based reparameterization to better capture latent randomness and constrain latent representations, making them more interpretable and controllable.

Several methods have been proposed to tackle clustering without requiring a predefined number of clusters. Deep adaptive clustering (DAC) [26] reframes clustering as a binary pairwise classification task, where the model learns to judge whether pairs of images belong to the same cluster. While DAC eliminates the need for predefined cluster numbers, it requires some labeled data for training and uses an alternating iterative learning algorithm to refine the model. AutoNovel [27] addresses discovering new classes in unlabeled data without needing predefined clusters. It employs self-supervised learning to pretrain a neural network on both labeled and unlabeled data, and then uses ranking statistics to generate pseudolabels for clustering. However, it still requires some labeled data during pretraining to learn feature representations. AutoClust [28] automates the selection of the optimal clustering algorithm and hyperparameters using cluster validity indices (CVIs). By combining multiple CVIs, AutoClust evaluates and selects the best algorithm and tunes hyperparameters to suit the dataset. While these methods do not require a predefined number of clusters to some extent, they still rely on labeled information during the training phase. In contrast, our model operates without any reliance on labeled data.

B. VAE and the Variants

Deep unsupervised clustering can be viewed as a specific aspect of the problem of disentangling latent variables. VAE [14] is an important generative model in that the latent space is structured. The encoder maps the samples to the latent variables in a low-dimensional space and then restores the latent variables to reconstructed samples through the decoding process. It assumes that the prior of latent variables is a Gaussian distribution. This assumption is based on computation convenience, without considering rationality, which will inevitably affect the generation ability of the model. Importance weighted autoencoder [29] is one of the most important improvement methods for VAE models. From the perspective of the variational lower bound, it improves the performance of the generative model by weakening the role of the encoder in the variational lower bound. Many deep generative models want to generate high-resolution image samples that will be combined with convolutional networks. Deep convolutional inverse graphics network [30] integrates convolutional layers into VAE. The structure of the deep convolutional inverse graphics network is the same as VAE. It replaces the encoder and decoder in the VAE from the original fully connected layers to the convolutional and deconvolutional layers. Adversarial autoencoders (AAEs) [31] is a generative model that applies the idea of adversarial learning to the VAE training process. The main difference between VAE and AAE is that VAE uses the KL divergence of the prior distribution and posterior distribution to constrain the hidden variables, while AAE employs an adversarial network at the latent variable layer to enforce the aggregated posterior to match

TABLE I
NOTATIONS AND THEIR DESCRIPTIONS IN THIS ARTICLE

Notations	Descriptions
x	The vector of an original data sample.
w_i	The mixture weight of the i th Gaussian component.
μ_i	The means of the i th Gaussian component.
Σ_i	The variance of the i th Gaussian component.
M	The mixture number of GMM.
x_i	The i th data sample.
z_i	Latent variable, the embedding of x_i .
c_i	Model variable, denotes which component has been chosen.
μ_x	The mean of distribution $p(x z)$.
σ_x	The variance of distribution $p(x z)$.
ϕ	The parameters of distribution $p(x z)$.
μ_z	The mean of distribution $p(z c)$.
σ_z	The variance of distribution $p(z c)$.
θ	The parameters of distribution $p(z c)$.
π	Model variable, indicates the probability of each component being chosen.
v	Variable of Dirichlet process.
α	Parameter of function $p(v)$.
$\tau_{k,1}$	Parameter of function of $q(v_k)$.
$\tau_{k,2}$	Parameter of function of $q(v_k)$.
μ_c	The mean of distribution $q(c x)$.
σ_c	The variance of distribution $q(c x)$.
ν	The parameters of distribution $q(c x)$.
μ_o	The mean of distribution $q(z x)$.
σ_o	The variance of distribution $q(z x)$.
δ	The parameters of distribution $q(z x)$.
N	The number of samples.
K	The number of Gaussian components.

a chosen prior distribution. This adversarial network consists of a generator and a discriminator. The generator of AAE uses the same neural network as the encoder to fake a distribution of the hidden variables that is close to the real distribution, and the discriminator is responsible for distinguishing the samples obtained from the true and false distributions.

The work that is most related to ours is GMVAE [12]. The main difference is that the GMVAE is based on the assumption that the observed data is generated from a GMMs. The performance is highly reliant on the mixture number setting. Our model is based on the assumption that the prior is an infinite Gaussian distribution. The mixture number is learned by the model. Therefore, it is also more stable and adaptable.

III. PRELIMINARY KNOWLEDGE

To better understand our proposed model, we introduce some preliminary knowledge here. We first introduce the GMM, which is a classic probability model and the basis of GMVAE, which is then introduced as the benchmark against which our proposed model is compared. The notations used in this article are summarized in Table I.

A. GMM

A GMM [32] is a parametric probability density function represented as a weighted sum of M Gaussian component densities

$$p(x|\lambda) = \sum_{i=1}^M w_i g(x|\mu_i, \Sigma_i) \quad (1)$$

where x represents a D-dimensional continuous-valued data vector; w_i denotes the mixture weights (which must be positive

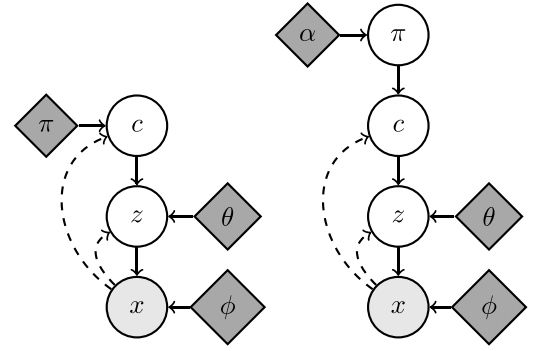


Fig. 2. Graphical models for GMVAE (left) and IGMVAE (right): the solid line represents the generative process and the dotted line represents the inference procedure.

and sum to one) and $g(x|\mu_i, \Sigma_i)$ represents the component Gaussian densities and can be written as follows:

$$g(x|\mu_i, \Sigma_i) = \frac{1}{(2\pi)^{D/2} |\Sigma_i|^{1/2}} \exp -\frac{1}{2} (x - \mu_i)' \Sigma_i^{-1} (x - \mu_i) \quad (2)$$

where μ_i and Σ_i are the component mean vector and covariance matrix. It is an extension of a single Gaussian probability density function and can smoothly approximate any shape of density distribution in theory. When applied to clustering, each Gaussian component can be regarded as a category according to different parameters of the Gaussian probability density function. Given a set of data points, the model parameters $\lambda = w_i, \mu_i, \Sigma_i, i = 1, \dots, M$ can be estimated. There are many parameter estimation methods, such as maximum likelihood and expectation maximization (EM) algorithms.

B. GMVAE

GMVAE is a benchmark deep unsupervised clustering method. It replaces the plain Gaussian prior of the latent space in VAE with GMM, as it is an intuitive extension of the unimodal Gaussian prior. If we assume that the observed data is generated from a GMM, inferring the category of a data point is equivalent to inferring which mode of the latent distribution the data point was generated from. The model is defined as follows:

$$c_i | \pi \sim \text{Multi}(\pi) \quad (3)$$

$$z_i | c_i, \theta \sim \mathcal{N}(\mu_z(c_i; \theta), \sigma_z(c_i; \theta)) \quad (4)$$

$$x_i | z_i, \phi \sim \mathcal{N}(\mu_x(z; \phi), \sigma_x(z; \phi)) \quad (5)$$

where sample x_i is generated from a neural network parameterized by ϕ and the latent variable z_i . z_i is sampled from a Gaussian mixture with means and variances specified by another neural network model parameterized by θ and with input c_i . Furthermore, c_i is a one-hot vector sampled from the mixing probability π , which chooses one component from the Gaussian mixture. We set the parameter $\pi_k = 1/K$ to make c_i uniformly distributed. The probability graph is depicted in Fig. 2.

IV. IGMVAE

In Section III, we learned GMVAE requires prior knowledge of the number of clusters to be fitted, while the IGMM permits

the automatic determination of the number of clusters. We replace the Gaussian prior of VAE with IGMM and introduce a novel clustering model named IGMVAE. The main idea is to introduce infinite components (i.e., clusters) into the model, and then the model can automatically decide which components are useful and which are unnecessary based on the data. Specifically, when a new sample arrives, it is assigned to an existing cluster with a certain probability, or a new cluster is created. Once this process is completed, all samples are partitioned into different clusters, and the number of clusters is determined without the need to predefine the number of clusters. Moreover, our model allows for cluster expansion dynamically. When a sample that does not belong to any existing cluster is encountered, the model has the ability to identify and generate a new cluster, enabling adaptive clustering. This infinite characteristic allows our proposed model to automatically determine the number of clusters based on the data, without requiring a predefined mixture number. Our model uses the DP to model the number of mixture components. The DP is a probabilistic process that enables us to stochastically generate clusters from an infinite number of components, each of which represents a Gaussian distribution. In this section, we discuss our proposed model's definition and the inference process.

A. Model Description

Our aim is to cluster a set of D -dimensional training samples $x_{n=1}^N$. For each training sample x , we learn a latent feature $z \in \mathbb{R}^{M \times 1}$ for it. We assume the latent features z follow infinite Gaussian mixtures. As there are infinite components, how to design the model to choose components from infinite mixtures becomes an important issue. We first introduce the model with a fixed number of mixtures, and then extend it to an infinite situation. The finite GMM with K components may be written as follows:

$$p(x|\lambda) = \sum_{j=1}^K \pi_j \mathcal{N}(\mu_j, s_j^{-1}) \quad (6)$$

where x is the training set, μ_j are the means, s_j are the precisions, π_j are the mixing proportions (which must be positive and sum to one) and $\mathcal{N}$ is a Gaussian with specified mean and variance. We introduce a latent variable c_i to represent the i th sample's cluster membership, so that $c_i = k$ denotes that the i th observation is assigned to the k th cluster. c_i is an important variable in simplifying the calculations. If we know which cluster each observation belongs to, then each sample's data likelihood is calculated from its cluster mean and covariance

$$p(x_i|c_i = k) = \mathcal{N}(x_i|\mu_k, \sigma_k^2). \quad (7)$$

The index c_i is selected from the mixture proportions π , representing the cluster membership of the i th sample

$$p(c_i|\pi) = \mathcal{C}(\pi), \text{ where } \sum_k \pi_k = 1. \quad (8)$$

The mixing proportions, π_k , are given a symmetric Dirichlet prior with concentration parameter α/K

$$p(\pi_1, \dots, \pi_K|\alpha) \sim \text{Dirichlet}(\alpha/K, \dots, \alpha/K) \quad (9)$$

where $\alpha/K = 1$ is for a flat Dirichlet distribution for simplicity. By assigning a flat Dirichlet distribution, it indicates that the model assigns equal weights to each component. The shape of the distribution is only affected by α because K is a fixed and known constant in a finite mixture model. π_k follows a Dirichlet distribution with its density function controlled by the parameter α .

Taking $K \rightarrow \infty$, then extends the model to an infinite mixtures situation. Instead of using a flat Dirichlet distribution for the construction of π , the construction process of π can use stick-breaking [33] to explain. The stick-breaking process is a constructive way to define a probability distribution over an infinite number of parameters.

The process of generating and selecting components (or clusters) is achieved through stick-breaking, which determines both the assignment of samples to existing clusters and the potential creation of new clusters.

Considering a "stick" of unit length, the stick is sequentially broken into segments, each representing the weight of a cluster. The break proportions are drawn from a Beta distribution

$$v_k|\alpha \sim \text{Beta}(1, \alpha), \pi_k = v_k \sum_{j=1}^{k-1} (1 - v_j) \quad (10)$$

where v_k represents the proportion of the stick allocated to the k th cluster. π_k is the weight of the k th cluster, which is calculated based on the lengths of all previous pieces. α is a concentration parameter that controls how likely it is for new clusters to be created.

When a new sample arrives, it has a certain probability of being assigned to an existing cluster. This probability is proportional to the size of each cluster π_k . Specifically, larger clusters (those with higher values of π_k) are more likely to receive new samples. This is akin to the "stick" being divided into smaller pieces: the larger the piece, the greater the probability that the new sample will be assigned to that cluster. If the probability for the new sample falls within a "new" section of the stick (i.e., the remaining portion that has not been allocated to any existing cluster), a new cluster is created. This process is analogous to the last part of the stick being "cut" and forming a new cluster. In this way, the model is able to dynamically generate new clusters as needed, whenever the existing clusters are not sufficient to accommodate new data. It allows for an infinite number of potential clusters, even though only a finite number are used at any given time. As more data is observed, more clusters may be "activated" as required by the data distribution.

Based on the above, our generative model is defined as follows:

$$p(x, z, c, \pi) = \prod_i^N [p(x_i|z_i)p(z_i|c_i)p(c_i|\pi)] \prod_k^K p(\pi_k) \quad (11)$$

where an observed sample x is generated in the following process:

$$v_k|\alpha \sim \text{Beta}(1, \alpha) \quad (12)$$

$$\pi_k = v_k \prod_{j=1}^{k-1} (1 - v_j) \quad (13)$$

for each sample x_i

$$c_i|\pi \sim \mathcal{C}(\{\pi\}) \quad (14)$$

$$z_i|c_i, \theta \sim \mathcal{N}(\mu_{z_i}(c_i, \theta), \sigma_z(c_i, \theta)) \quad (15)$$

$$x_i|z_i, \phi \sim \mathcal{N}(\mu_x(z_i, \phi), \sigma_x(z_i, \phi)) \quad (16)$$

where N is the number of samples. K is the number of Gaussian components. It is theoretically intended to be $K = \infty$. In order to obtain a closed-form solution in our subsequent calculations, we use a large value to represent infinity. Where μ_z, σ_z, μ_x , and σ_x are given by neural networks with parameters θ and ϕ , respectively. That is, the observed sample x_i is generated from a neural network observation model parameterized by ϕ and the latent variable z_i . Furthermore, the distribution of $p(z_i|c_i)$ is defined by another neural network model parameterized by θ and with input c_i . c_i is a one-hot vector sampled from a categorical distribution $\mathcal{C}(\cdot)$ parameterized by π , which indicates which component has been chosen. The variable π can be constructed by stick-breaking. The generation probability graph is shown in Fig. 2.

B. Variational Inference

Directly solving the generative model, which involves the finding of the maximum a posteriori (MAP) of latent variables and maximum likelihood estimation (MLE) for parameters, is difficult. To solve this issue, we use the variational inference method, that is, to find a distribution q that is easy to express and solve. When the difference between q and p is very small, q can be used as the approximate distribution of p and become the output result. Moreover, it is well known that reparameterization techniques commonly used for VAEs cannot be directly applied to discrete variables. Several possibilities for estimating the gradients of discrete variables have been proposed [15], [34], [35], [36]. Instead, we show that by adapting the architecture of a standard VAE, our estimator for the variational lower bound of our model can be optimized using standard backpropagation via a reparameterization trick, thus preserving the inference process more simply. The variational probability distributions are defined as follows:

$$q(v_k) = \text{Beta}(\tau_{k,1}, \tau_{k,2}), \text{ for } k = 1 : K - 1 \quad (17)$$

$$q(\pi_k) = v_k \prod_{i=1}^{k-1} (1 - v_i) \quad (18)$$

for each image x_i

$$q(c_i|x_i) = \mathcal{C}(\mu_c(z_i, \nu), \sigma_c(z_i, \nu)) \quad (19)$$

$$q(z_i|x_i) = \mathcal{N}(\mu_o(x_i, \delta), \sigma_o(x_i, \delta)) \quad (20)$$

where z_i is estimated by a neural network model $\mathcal{N}(\cdot)$ parameterized by δ and x_i . c_i is estimated by a neural network model $\mathcal{C}(\cdot)$ parameterized by ν and c_i . While c_i can be equivalent to label information, which is discrete information. The sampling process of discrete data from a categorical distribution is not differentiable, meaning that backpropagation will not work. Therefore, we set the last layer of this neural network as a Gumbel-softmax [37], which is a continuous distribution that

approximates samples from a categorical distribution and also works with backpropagation.

Our model is trained with the variational inference objective of the log-evidence lower bound (ELBO), which can be written as follows:

$$\text{ELBO} = \mathbb{E}_q \log p(x, z, c, \pi) - \mathbb{E}_q \log q(z, c, \pi|x). \quad (21)$$

Applying the independence assumption, we get the decomposition

$$p(x, z, c, \pi) = \prod_i^N p(x_i|z_i) p(z_i|c_i) p(c_i|\pi) \prod_k^\infty p(\pi_k|\alpha) \quad (22)$$

and

$$q(z, c, \pi|x) = \prod_i^N q(z_i|x_i) q(c_i|x_i) \prod_k^\infty q(\pi_k|\tau). \quad (23)$$

Then, (21) can be written as follows:

$$\begin{aligned} \text{ELBO} &= \mathbb{E}_q \log p(x|z) \\ &\quad - [\mathbb{E}_q \log q(z|x) - \mathbb{E}_q \log p(z|c)] \\ &\quad - [\mathbb{E}_q \log q(c|x) - \mathbb{E}_q \log p(c|\pi)] \\ &\quad - [\mathbb{E}_q \log q(\pi|\tau) - \mathbb{E}_q \log p(\pi|\alpha)] \\ &= \mathbb{E}_q [\log p(x|z)] - \text{KL}[q(z|x)||p(z|c)] \\ &\quad - \text{KL}[q(c|x)||p(c|\pi)] - \text{KL}[q(\pi)||p(\pi)]. \quad (24) \end{aligned}$$

We refer to the terms in the ELBO as the reconstruction loss term $\mathbb{E}_q \log p(x|z)$, z KL-divergence loss term, c KL-divergence loss term, and π KL-divergence loss term, respectively.

The reconstruction loss term can be computed by the corresponding loss function, e.g., mean squared error (MSE) and binary cross entropy (BCE).

As $q(z|x)$ and $p(z|c)$ are two Gaussian distributions, z KL-divergence loss term can be calculated by the KL distance formula of two Gaussian distributions. For two Gaussian distributions p and q with mean μ_1, μ_2 and variance σ_1^2, σ_2^2 , respectively, the KL divergence calculation formula is

$$\text{KL}[p, q] = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}. \quad (25)$$

Putting (15) and (20) into (25), then we can get

$$\text{KL}[q(z|x)||p(z|c)] = \log \frac{\sigma_z}{\sigma_o} + \frac{\sigma_o^2 + (\mu_o - \mu_z)^2}{2\sigma_z^2} - \frac{1}{2} \quad (26)$$

where μ_o and σ_o are the mean and variance of $q(z|x)$ that are given by neural networks. μ_z and σ_z are the means and variance of $p(z|c)$ that is given by another neural network.

The c KL-divergence loss term can be calculated by the KL distance formula. Details derivations are long, so they are described in the supplementary section. We show the conclusion here

$$\begin{aligned} &\text{KL}[q(c|x)||p(c|\pi)] \\ &= \mathbb{E}_{q(c, \pi)} \left[\log \frac{q(c)}{p(c)} \right] \end{aligned}$$

$$\begin{aligned}
&= \sum_i \sum_{k=1}^K q(c|x)_{i,k} [\log q(c|x)_{i,k}] \\
&\quad - \sum_i \sum_{k=1}^K q(c|x)_{i,k} [\psi(\tau_{k,1}) - \psi(\tau_{k,1} + \tau_{k,2}) \\
&\quad - \sum_{j=1}^{k-1} (\psi(\tau_{j,2}) - \psi(\tau_{j,1} + \tau_{j,2}))], \text{ where } \tau_{K,1} = 1, \tau_{K,2} = 0.
\end{aligned} \tag{27}$$

In the calculation process, we will set a large number K , which represents the upper limit of the cluster number. Here $q(c|x)_{i,k}$ is the $q(c|x)$ k th dimension value in i th sample. $\psi(\cdot)$ is a psi function.

The π KL-divergence loss term derivation details can be found in the supplementary section. We show the last calculation steps in the following equation:

$$\text{KL}[q(\pi)||p(\pi)] = \sum_{k=1}^K \mathbb{E}_q \log q(\pi_k) - \sum_{k=1}^K \mathbb{E}_q \log p(\pi_k) \tag{28}$$

where

$$\begin{aligned}
\mathbb{E}_q \log p(\pi_k) &= \mathbb{E}_q \left[\log p(v_k) + \sum_{j=1}^{k-1} \log p(1 - v_j) \right] \\
&= (\alpha - 1)(\psi(\tau_{k,2}) - \psi(\tau_{k,2} + \tau_{k,1})) \\
&\quad + \sum_{j=1}^{k-1} [(\alpha - 1)(\psi(\tau_{j,2}) - \psi(\tau_{j,2} + \tau_{j,1})) \\
&\quad - \log \mathbf{B}(1, \alpha) - \log \mathbf{B}(\alpha, 1)] \tag{29}
\end{aligned}$$

and

$$\begin{aligned}
\mathbb{E}_q \log q(\pi_k) &= \mathbb{E}_q \left[\log q(v_k) + \sum_{j=1}^{k-1} \log q(1 - v_j) \right] \\
&= (\tau_{k,1} - 1) [\psi(\tau_{k,1}) - \psi(\tau_{k,1} + \tau_{k,2})] \\
&\quad + (\tau_{k,2} - 1) [\psi(\tau_{k,2}) - \psi(\tau_{k,2} + \tau_{k,1})] \\
&\quad + \sum_{j=1}^{k-1} [(\tau_{j,1} - 1) [\psi(\tau_{j,1}) - \psi(\tau_{j,1} + \tau_{j,2})] \\
&\quad + (\tau_{j,2} - 1) [\psi(\tau_{j,2}) - \psi(\tau_{j,2} + \tau_{j,1})] \\
&\quad - \log \mathbf{B}(\tau_{k,1}, \tau_{k,2}) - \log \mathbf{B}(\tau_{j,2}, \tau_{j,1})] \tag{30}
\end{aligned}$$

where $\psi(\cdot)$ is the psi function. $\mathbf{B}$ is a Beta function parameterized by α .

The above is the calculation process of the ELBO that optimizes our model in the following experiments.

C. Optimization

Our optimization goal is to maximize ELBO. We can use four loss terms in the training stage, which are $\mathcal{L}_{\text{rec}}$, $\mathcal{L}_{\text{gaus}}$, $\mathcal{L}_{\text{categ}}$, and $\mathcal{L}_{\text{stickbreaking}}$, corresponding to the reconstruction loss term, z KL-divergence loss term, c KL-divergence loss term and π KL-divergence loss term corresponding to ELBO

Algorithm 1: Training Process of IGMVAE.

Input: Training samples $x_{i_{n=1}}^N$, batch-size m , IGMM parameter α , K , loss function weight w_{rec} , w_{gaus} , w_{categ} , $w_{\text{stickbreaking}}$.

Output: cluster index $\{c_n\}_{n=1}^N$, neural network parameters ϕ and θ

while not converge **do** Draw a mini-batch training data $\{x_i\}$;
 Feed forward x_i to $q(z_i|x_i)$ and $q(c_i|x_i)$, and get latent variable z_i and c_i ;

Feed forward latent variable z_i and c_i to compute $p(x_i|z_i)$ and $p(z_i|c_i)$;

Evaluate the whole loss $\mathcal{L}$ via (31);

Update network parameters θ , ϕ , ν and δ ;

Update IGMM parameter τ .

end while

terms, respectively. The whole loss can be written as follows:

$$\begin{aligned}
\mathcal{L} &= -w_{\text{rec}}\mathcal{L}_{\text{rec}} + w_{\text{gaus}}\mathcal{L}_{\text{gaus}} + w_{\text{categ}}\mathcal{L}_{\text{categ}} \\
&\quad + w_{\text{stickbreaking}}\mathcal{L}_{\text{stickbreaking}} \tag{31}
\end{aligned}$$

where w_{rec} , w_{gaus} , w_{categ} , and $w_{\text{stickbreaking}}$ are weight. They are hyper parameters to balance each part of the whole loss. Below we give a pseudocode showing how the updates are performed on the IGMVAE model. As shown in Algorithm 1, the parameters are updated by optimizing the whole loss.

V. EXPERIMENTS

This article includes two sets of experiments: In this section, we evaluate: 1) whether our model learns a meaningful category discriminable latent space and does an effective clustering adaptively; and 2) whether our model can do category-based generation in an unsupervised mode.

A. Unsupervised Clustering

1) *Setting and Datasets:* The whole model architecture is shown in Fig. 3. It contains an encoder and a decoder. The encoding process consists of two neural networks.

a) The label information z generated model

$$q(z_i|x_i) = \mathcal{C}(\mu_o(x_i, \nu), \sigma_o(x_i, \nu)).$$

b) The hidden variable x generated model

$$q(c_i|x_i) = \mathcal{N}(\mu_c(x_i, \delta), \sigma_c(x_i, \delta))$$

where $\mathcal{C}(\cdot)$ is Gumbel-softmax function [37].

The decoding process consists of two neural networks.

a) The sample x reconstructed model

$$p(x_i|z_i) = \mathcal{N}(\mu_x(z_i, \phi), \sigma_x(z_i, \phi)).$$

b) The hidden variable z reconstructed model

$$p(z_i|c_i) = \mathcal{N}(\mu_z(c_i, \theta), \sigma_z(c_i, \theta)).$$

We train our models on a single NVIDIA A6000 GPU. In our experiments, we used the Adam optimizer with a learning rate of 0.0001 and trained the model for 100 steps. Detailed training settings are provided in Table II.

We evaluate our model on two image datasets and one network traffic dataset.

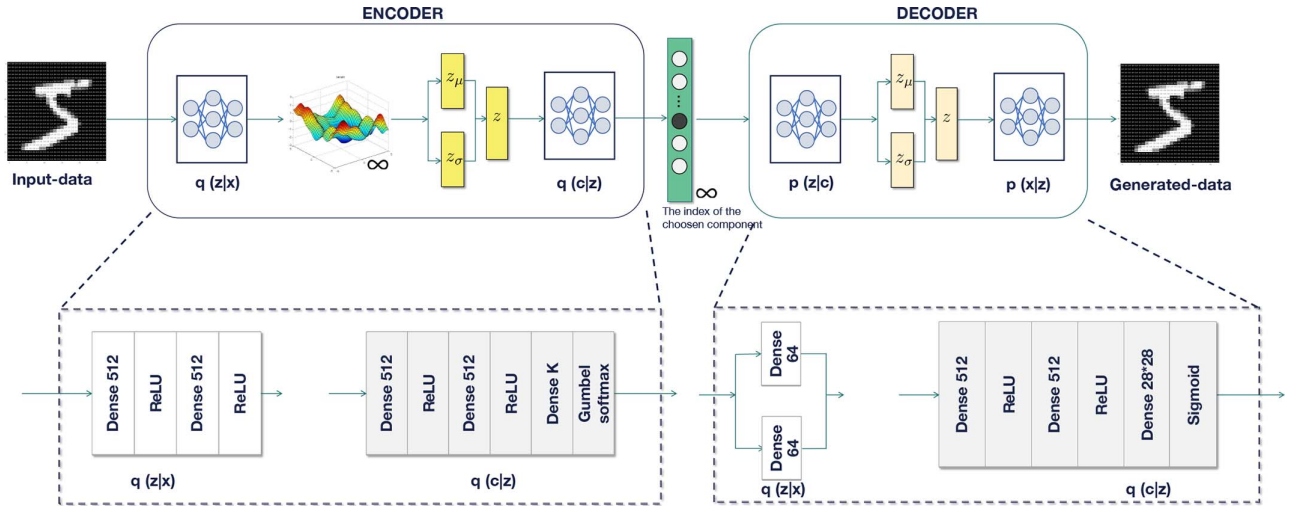


Fig. 3. Infinite Gaussian mixture autoencoder.

TABLE II
TRAINING DETAILS

Parameter	Value
Epoch	100
Batch Size	64
Learning Rate	0.0001
Optimizer	Adam
w_{rec}	1
w_{gaus}	1
w_{categ}	1
$w_{stickbreaking}$	1
Reconstruct Loss Type	Binary Cross Entropy
K	500
α	1
$q(z x)$	512—RELU—512—RELU—64
$q(c z)$	512—RELU—512—RELU—500—Gumble Softmax
$p(z c)$	64
$p(x z)$	512—RELU—512—RELU—28*28

- MNIST**: The MNIST dataset is a large-scale 28-by-28 pixel handwritten digit database, containing a training set of 60 000 samples and a test set of 10 000 samples [38].
- FashionMNIST**: Fashion-MNIST is an image dataset that covers a total of 70 000 different product front images from ten categories. The size, format, and train/test split of Fashion-MNIST are the same as the MNIST [39].
- YouTube Traffic**: YouTube Traffic is a YouTube video wireless traffic dataset that contains ten video content categories. Each category includes 300 samples. Each sample is a 500 by 1 vector. The training set and test set are divided in a ratio of 7:3 [40].

2) **Comparative Methods**: Our model is an improvement of GMVAE. Following [13], we use the same network architecture. We compare our model with GMVAE and show the comparison results with the different number of mixtures given by GMVAE.

3) **Evaluation Metrics**: The clustering evaluation metrics are as follows.

- Unsupervised Clustering Accuracy (ACC)**: ACC is the unsupervised equivalent of classification accuracy, provided by the Hungarian algorithm [41]. It uses a mapping function to find the best mapping between the cluster assignment label and the ground truth label. This mapping is required because an unsupervised algorithm may use a different label rather than the actual ground truth label to represent the same cluster. The implementation details can be found in Xu et al. [42].
- Normalized Mutual Information (NMI)**: NMI [42] is an information-theoretic metric that measures the mutual information between cluster assignments and the ground truth labels. It is normalized by the average entropy of both ground labels and cluster assignments.
- Adjusted Rand Index (ARI)**: The Rand Index [43] computes a similarity measure between two clusters by considering all pairs of samples and counting pairs that are assigned in the same or different clusters in the predicted and true clustering. The ARI is the corrected-for-chance version of the Rand Index.

4) **Results**: We evaluate our model's ability to represent discrete information in the data in a clustering task. We compare our model with GMVAE which is also based on the VAE framework and has a good clustering effect, but the performance is unstable with different cluster number settings. In this experiment, we set the GMVAE cluster numbers to 5, 10, 25, and 100 to observe the clustering performance with different cluster number settings. For our model, it defines infinite mixtures in advance (generally, we can set a large number K to represent infinity). It automatically determines which mixtures are useful and which are unnecessary based on the data.

The model automatically determines the cluster number is 20/18/11 on MNIST, FashionMNIST, and YouTube Traffic, respectively. Of these three datasets, the number of clusters learned by our model on YouTube traffic data is closest to the ground truth. As shown in Fig. 1, its data is highly clustered,

TABLE III
CLUSTERING PERFORMANCE OF DIFFERENT MODELS ON MNIST/FASHIONMNIST/YOUTUBE TRAFFIC IN ACC
(HIGHER IS BETTER), NMI (HIGHER IS BETTER), AND ARI (HIGHER IS BETTER)

	MNIST			FashionMNIST			Traffic		
	ACC ↑	NMI ↑	ARI ↑	ACC ↑	NMI ↑	ARI ↑	ACC ↑	NMI ↑	ARI ↑
GMVAE (#cluster=5)	0.44	0.46	0.32	0.44	0.51	0.37	0.43	0.47	0.30
GMVAE (#cluster=10)	0.64	0.62	0.52	0.69	0.69	0.58	0.52	0.52	0.35
GMVAE (#cluster=25)	0.44	0.63	0.41	0.50	0.60	0.39	0.31	0.53	0.27
GMVAE (#cluster=100)	0.16	0.57	0.16	0.51	0.61	0.40	0.13	0.48	0.11
Our	0.54	0.49	0.37	0.59	0.68	0.53	0.49	0.45	0.30

the data in the same cluster is very close, and the direct distance between different clusters is relatively far.

For a fair comparison with GMVAE, we use the same neural network architecture and hyperparameters. A summary of clustering results is shown in Table III. From Table III, we can see that the clustering performance of GMVAE is heavily dependent on the cluster number we set in the training process. When the given cluster number is the same as the ground truth (10 for MNIST, 10 for FashionMNIST, and 10 for YouTube traffic), the performance is the best. The bigger the gap between the given cluster number and the ground truth, the worse the performance. For our model, the clustering performance is slightly worse than GMVAE when the given cluster number is the ground truth, but better than GMVAE in other cases. This is because IGMVAE dynamically discovers the optimal number of components based on the data, and the number of components it discovers may differ from the true number. When the predefined cluster number in GMVAE matches the true number of clusters, its performance may outperform IGMVAE. However, when the predefined cluster number deviates significantly from the true number, IGMVAE outperforms GMVAE.

In order to better visualize the effect of clustering, we map the latent variables to a 2-D data space using t-SNE. As shown in Fig. 1, we can observe that the clustering ability of GMVAE is variable and unstable with the different number of clusters given.

As we introduced before, our model can adapt novel categories' data, that is, the data that do not belong to the original training dataset can be assigned to a new cluster. We did an incremental experiment to test the adaptability of our model to the novel categories of data.

In this experiment, we separate the dataset into two parts. One is a base dataset (classes 0 to 6), and the other is an incremental dataset (classes 7 to 9). This experiment consists of two stages. In the first stage, we feed the base dataset, and the cluster number is learned by the model. In the second stage, we feed the incremental dataset, and the new cluster number is learned in this stage. In the base training stage, the cluster number is 13/14/8 on MNIST/Fashion MNIST/YouTube Traffic, respectively. In the incremental stage, the cluster number is 18/18/11, respectively. The number of clusters has increased, which means that new clusters are expanded on the basis of base training. Fig. 4 shows the generated images when the novel categories of data are added. We can see that the model can be dynamically compatible with the novel categories.

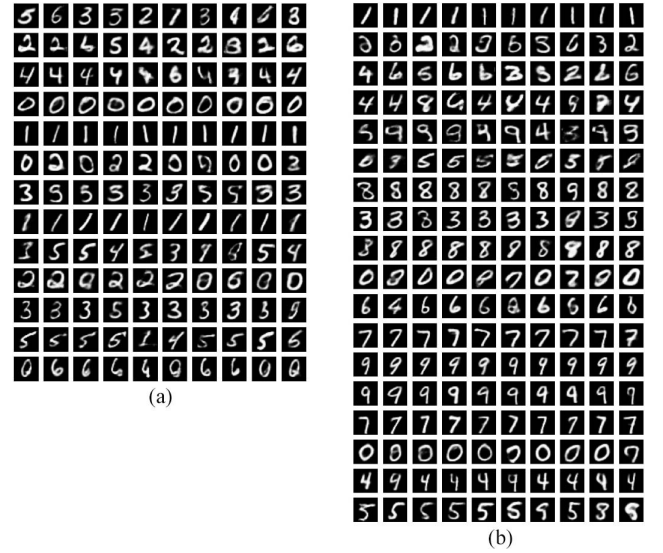


Fig. 4. MNIST digital generation. (a) Generated images in base training stage (classes 0 to 6). (b) Generated images when novel categories sample added in (classes 7 to 9).

B. Data Generation

1) *Setting and Datasets*: We did a generation experiment on the MNIST [38] dataset.

2) *Evaluation Metrics*: For evaluation, we use the following metrics.

- bits/dim*: Bits per dimension [44] converts the negative number of the maximum likelihood value to measure the quality of the generated data. The computation formula is

$$\text{bits/dim} = -(\log(\text{likelihood}/(\text{height} \times \text{weight} \times \text{channel})) - \ln(256/2))/\ln 2.$$
- Inception Score (IS)*: The IS¹ is computed by inputting the generated image into the classification model preprocessed by the real image to measure its distance from the real distribution.
- FID*: FID² reflects the distance between the generated image and the real image. It measures the distance between two multivariate normal distributions. The compute formula is as follows:

¹<https://www.techtarget.com/searchenterpriseai/definition/inception-score-IS>

²<https://www.techtarget.com/searchenterpriseai/definition/Frechet-inception-distance-FID>

TABLE IV
GENERATION PERFORMANCE OF DIFFERENT MODELS ON MNIST/FASHIONMNIST/YOUTUBE TRAFFIC IN FID
(HIGHER IS BETTER), IS (LOWER IS BETTER), AND BITS/DIM (LOWER IS BETTER)

	MNIST			FashionMNIST			Traffic		
	FID ↓	IS ↑	bits/dim ↓	FID ↓	IS ↑	bits/dim ↓	FID ↓	IS ↑	bits/dim ↓
GMVAE (#cluster=5)	59.24	1.65	22.50	103.11	1.80	29.08	-	-	15.10
GMVAE (#cluster=10)	58.95	1.54	22.20	103.94	1.71	29.19	-	-	15.10
GMVAE (#cluster=25)	53.63	1.45	22.30	100.59	1.58	29.33	-	-	15.48
GMVAE (#cluster=100)	54.49	1.70	22.64	98.84	1.80	29.24	-	-	15.23
Our	57.21	1.41	24.79	105.63	1.48	12.88	-	-	14.94

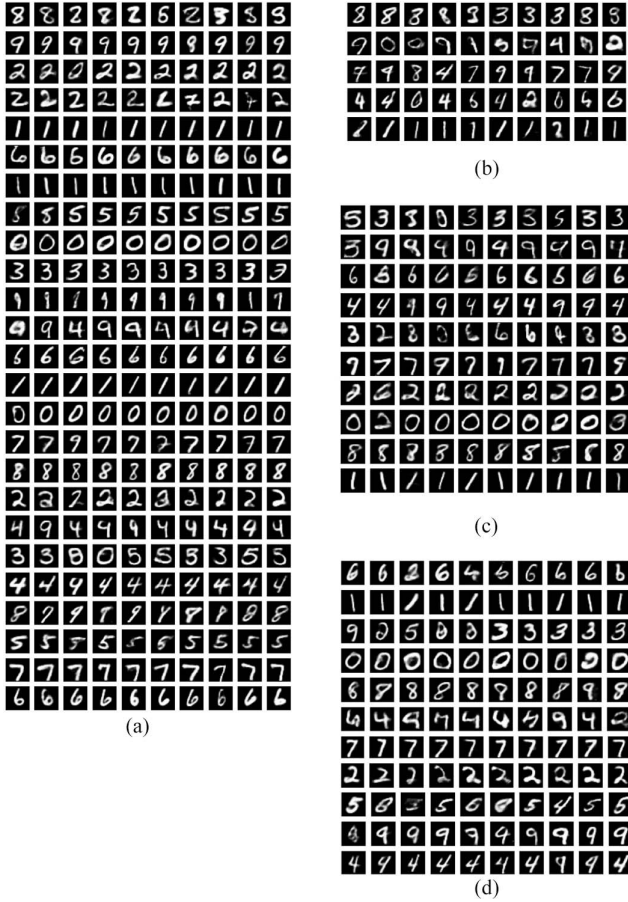


Fig. 5. MNIST digital generation (each row is generated by one cluster). (a) Generated images with GMVAE with mixture number setting 25, same digital images are generated by different clusters. (b) Generated images with GMVAE with mixture number setting 10. (c) Generated images with GMVAE with mixture number setting 5, multiple digital images are generated by one cluster. (d) Generated images with IGMVAE, different digital images are generated by different clusters.

$$FID = \|\mu_{\text{real}} - \mu_{\text{gen}}\|^2 + Tr(\Sigma_{\text{real}} + \Sigma_{\text{gen}} - 2(\Sigma_{\text{real}}\Sigma_{\text{gen}})^{1/2})$$

where μ_{real} and Σ_{real} are the mean and covariance of the real image's feature value, respectively, and μ_{gen} and Σ_{gen} are the mean and covariance of the generative image's feature value, respectively.

3) *Results:* Our proposed model can naturally be used to generate images. Our model can adaptively learn separable latent variables according to the training data. Sample from different Gaussian distributions for generation, it can perform data generation by category. Since GMVAE needs to specify

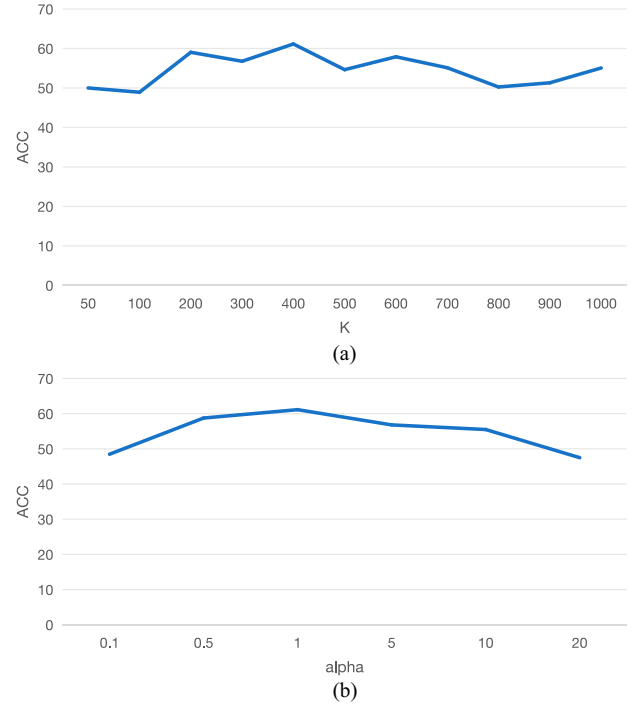


Fig. 6. Impact of K and α on model performance. (a) Accuracy on the MNIST dataset with different values of K. (b) Accuracy on the MNIST dataset with different values of α .

the number of clusters. If the given number of clusters is less than the ground-truth cluster number, multiple categories of data may be generated in one Gaussian distribution. In contrast, the samples generated from multiple distributions will appear in one category of images. In Fig. 5, we can observe that if the cluster number is set to 25, then different clusters will generate images of the same figure, such as the digits 1 and 2. In Fig. 5, we pick the top 10 probability cluster to generate images. We can observe that our model can generate similar samples of the same cluster and different clusters generate different category images.

Furthermore, we test our image generation quality on three datasets and the results are shown in Table IV. Our model is comparable to GMVAE, outperforming it in some cases and underperforming it in others. This is because our generation framework follows the same architecture as GMVAE without introducing additional parameters to enhance generation quality. The results show that our model can extend to an infinite number of components while achieving categorywise generation, with minimal impact on generation quality.

C. Ablation Study

We evaluate the clustering performances of our method with respect to K and α . K represents the upper limit on the number of Gaussian components, which is theoretically infinite. However, we explored the effect of setting a finite upper bound for K . α controls the tightness of clustering and influences the probability of assigning new data points to existing clusters versus creating new clusters. The result on MNIST is shown in Fig. 6. We found that setting the upper limit of the component number K to be above 200 gets relatively good results. Regarding the selection of α , we found that $\alpha = 1$ give the best results overall.

VI. CONCLUSION

In this article, we proposed a novel deep unsupervised clustering model based on a generative framework VAE, namely IGMVAE, in which we replace the latent space prior to a single normal distribution with an infinite Gaussian distribution. We evaluate our model's clustering ability using three datasets and show that our model has good clustering performance and can adaptively adjust clusters based on observed data. At the same time, when new categories of data are added, the model can adaptively expand new clusters. Moreover, as our model is based on a generated framework, it has the ability to generate images. As we restricted the latent space of the observed data with IGMM prior, it can naturally generate different categories of data in an unsupervised manner.

Although our model achieves adaptive clustering and generation to some extent, there is still room for improvement, especially when confronted with datasets exhibiting skewed and multimodal cluster distributions. These challenges arise when the underlying data structure is more complex and difficult to model. In the future, we will explore more realistic priors for the latent space that better capture the true distributions of real-world data. In addition, our current model is based on VAE, we will extend the idea of infinite components to more powerful generative models, such as diffusion models, which could potentially offer improved performance in capturing complex data structures and generate more realistic samples.

REFERENCES

- [1] H. Saeedi Emadi and S. M. Mazinani, "A novel anomaly detection algorithm using DBSCAN and SVM in wireless sensor networks," *Wireless Pers. Commun.*, vol. 98, no. 2, pp. 2025–2035, Apr. 2018.
- [2] S. Wibisono, M. Anwar, A. Supriyanto, and I. Amin, "Multivariate weather anomaly detection using DBSCAN clustering algorithm," *J. Phys. Conf. Ser.*, vol. 1869, no. 1, 2021, Art. no. 012077.
- [3] H. Tang, K. Chen, and K. Jia, "Unsupervised domain adaptation via structurally regularized deep clustering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8725–8735.
- [4] Q. Zhou et al., "Cluster adaptation networks for unsupervised domain adaptation," *Image Vis. Comput.*, vol. 108, 2021, Art. no. 104137.
- [5] Y. Meng, Y. Zhang, J. Huang, Y. Zhang, C. Zhang, and J. Han, "Hierarchical topic mining via joint spherical tree and text embedding," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1908–1917.
- [6] J. Yang, D. Parikh, and D. Batra, "Joint unsupervised learning of deep representations and image clusters," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 5147–5156.
- [7] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A k-means clustering algorithm," *J. Roy. Statist. Soc. Series c (Applied Statistics)*, vol. 28, no. 1, pp. 100–108, 1979.
- [8] M. Ester et al., "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, vol. 96, no. 34, 1996, pp. 226–231.
- [9] D. A. Reynolds and R. C. Rose, "Robust text-independent speaker identification using Gaussian mixture speaker models," *IEEE Trans. Speech Audio Process.*, vol. 3, no. 1, pp. 72–83, Jan. 1995.
- [10] Y. Ren et al., "Deep clustering: A comprehensive survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 4, pp. 5858–5878, Apr. 2024.
- [11] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 132–149.
- [12] N. Dilokthanakul et al., "Deep unsupervised clustering with Gaussian mixture variational autoencoders," 2016, *arXiv:1611.02648*.
- [13] D. P. Kingma, S. Mohamed, D. Jimenez Rezende, and M. Welling, "Semi-supervised learning with deep generative models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 3581–3589.
- [14] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.
- [15] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2014, pp. 1278–1286.
- [16] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2016, pp. 478–487.
- [17] W. Hu, T. Miyato, S. Tokui, E. Matsumoto, and M. Sugiyama, "Learning discrete representations via information maximizing self-augmented training," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1558–1567.
- [18] U. Shaham, K. Stanton, H. Li, B. Nadler, R. Basri, and Y. Kluger, "SpectralNet: Spectral clustering using deep neural networks," in *Proc. 6th Int. Conf. Learn. Representations (ICLR)*, 2018, pp. 1–12.
- [19] L. Yang, N.-M. Cheung, J. Li, and J. Fang, "Deep clustering by Gaussian mixture variational autoencoders with graph embedding," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 6440–6449.
- [20] X. Li, Z. Chen, L. K. Poon, and N. L. Zhang, "Learning latent superstructures in variational autoencoders for deep multidimensional clustering," 2018, *arXiv:1803.05206*.
- [21] V. Prasad, D. Das, and B. Bhowmick, "Variational clustering: Leveraging variational autoencoders for image clustering," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1–10.
- [22] L. Yang, W. Fan, and N. Bouguila, "Deep clustering analysis via dual variational autoencoder with spherical latent embeddings," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 6303–6312, Sep. 2023.
- [23] F. Ye and A. G. Bors, "Deep mixture generative autoencoders," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5789–5803, Oct. 2022.
- [24] Q. Ji, Y. Sun, J. Gao, Y. Hu, and B. Yin, "A decoder-free variational deep embedding for unsupervised clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5681–5693, Oct. 2022.
- [25] Y. Liu et al., "Cloud-VAE: Variational autoencoder with concepts embedded," *Pattern Recognit.*, vol. 140, 2023, Art. no. 109530.
- [26] J. Chang, L. Wang, G. Meng, S. Xiang, and C. Pan, "Deep adaptive image clustering," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5879–5887.
- [27] K. Han, S.-A. Rebuffi, S. Ehrhardt, A. Vedaldi, and A. Zisserman, "AutoNovel: Automatically discovering and learning new visual categories," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 6767–6781, Nov. 2021.
- [28] Y. Poulakis, C. Doukeridis, and D. Kyriazis, "AutoClust: A framework for automated clustering based on cluster validity indices," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1220–1225.
- [29] Y. Burda, R. Grosse, and R. Salakhutdinov, "Importance weighted autoencoders," 2015, *arXiv:1509.00519*.
- [30] T. D. Kulkarni, W. F. Whitney, P. Kohli, and J. Tenenbaum, "Deep convolutional inverse graphics network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2539–2547.
- [31] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey, "Adversarial Autoencoders," 2015, *arXiv:1511.05644*.
- [32] D. A. Reynolds et al., "Gaussian mixture models," *Encyclopedia Biometrics*, vol. 741, pp. 659–663, 2009.

- [33] J. Sethuraman, "A constructive definition of Dirichlet priors," *Statistica Sinica*, vol. 4, no. 2, pp. 639–650, 1994.
- [34] P. W. Glynn, "Likelihood ratio gradient estimation for stochastic systems," *Commun. ACM*, vol. 33, no. 10, pp. 75–84, 1990.
- [35] T. R. Aueb et al., "Local expectation gradients for black box variational inference," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2638–2646.
- [36] A. Graves, "Stochastic backpropagation through mixture density distributions," 2016, *arXiv:1607.05690*.
- [37] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-softmax," 2016, *arXiv:1611.01144*.
- [38] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [39] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [40] Y. Li et al., "Deep content: Unveiling video streaming content from encrypted WiFi traffic," in *Proc. IEEE 17th Int. Symp. Netw. Comput. IEEE Int. Symp. Spread Spectr. Tech. Appl. (NCA)*, 2018, pp. 1–8.
- [41] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logist. Quart.*, vol. 2, nos. 1–2, pp. 83–97, 1955.
- [42] W. Xu, X. Liu, and Y. Gong, "Document clustering based on non-negative matrix factorization," in *Proc. 26th Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2003, pp. 267–273.
- [43] E. Zuccarelli, "Performance metrics in machine learning—Part 3: Clustering," *Towards Data Science*, 2021. Accessed: Mar. 10, 2026. [Online]. Available: <https://towardsdatascience.com/performance-metrics-in-machine-learning-part-3-clustering-d69550662dc6>
- [44] A. Van den Oord et al., "Conditional image generation with pixelCNN decoders," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, Dec. 2016, pp. 4790–4798.



Ying Li received the B.E. and M.S. degrees in software engineering from Taiyuan University of Technology, Taiyuan, China in 2014 and 2017, respectively, and the Ph.D. degree in computer science school from the University of Technology Sydney, Ultimo, NSW, Australia, in 2023.

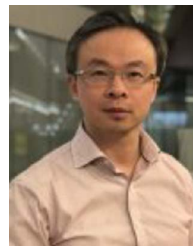
She is currently a Lecturer with the School of Artificial Intelligence and Computer Science, North China University of Technology, Beijing, China. Her research interests include time-aware deep learning and Bayesian nonparametrics.



Junyu Xuan (Senior Member, IEEE) received Ph.D. degree in computer science from UTS. He is currently a Senior Lecturer with the (AAIL), University of Technology Sydney (UTS), Australia. He has authored or coauthored more than 40 articles in high-quality journals and conferences, including the *Artificial Intelligence Journal*, *Machine Learning Journal*, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON FUZZY SYSTEMS, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA

ENGINEERING, IEEE TRANSACTIONS ON CYBERNETICS, *ACM TOIS*, *ACM TIST*, *ACM Computing Surveys*, *ICDM*, *NeurIPS*, *AAAI*, and *IJCNN*. His research interests include machine learning, Bayesian nonparametric learning, text mining, and web mining. He was a PC member of conferences, such as NIPS, ICLR, IJCAI, and IJCNN.

Dr. Xuan is an ARC Discovery Early Career Researcher Award (DECRA) fellow. His research interests include machine learning, Bayesian nonparametrics, and data mining.



Richard Yida Xu (Member, IEEE) received Bachelor's degree in computer engineering from UNSW Australia, in 2001, and the Ph.D. degree in computer science from the University of Technology Sydney (UTS), in 2006. He is currently a Full Professor with the Department of Mathematics, Hong Kong Baptist University (HKBU), Hong Kong. He has authored about 50 articles, including IEEE TRANSACTION ON CYBERNETICS, IEEE TRANSACTION ON IMAGE PROCESS, IEEE TRANSACTION ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTION ON

NEURAL NETWORKS AND LEARNING SYSTEMS, and *Pattern Recognition*. His research interests include machine learning, AI, Bayesian nonparametrics, and learning theory.